

Руководство администратора

Эксплуатация production-контура: архитектура, первый запуск, повседневные операции, обновления через `apply-update.bat`, резервные копии, пользователи и типовые сбои.

Версия документа: 1.0.0 · июль 2026 · Без иллюстраций

Содержание

1. Архитектура
2. Первый запуск
3. Повседневные операции
4. Обновления
5. Резервные копии и восстановление
6. Пользователи и права
7. Файл `backend/.env`
8. Страницы ошибок
9. Диагностика
10. Эскалация разработчику

1. Архитектура

Production-контур — три контейнера Docker и PostgreSQL на хосте Windows:

Сервис	Порт	Роль
frontend	5173	Веб-интерфейс (собранный статика, <code>vite preview</code>)
backend	8000	API и Django Admin (Gunicorn)
tileserver	8080	Векторная карта из <code>map.mbtiles</code>
PostgreSQL	5432	БД на хосте (<code>DB_HOST=host.docker.internal</code>)

Запуск всегда через два файла compose: `docker-compose.yml` + `docker-compose.server.yml`. Второй отключает bind-mount исходников frontend и поднимает production-образ.

Ветка git на сервере — только **production**. Разработка ведётся на `develop_report` (и других ветках) на машине разработчика.

2. Первый запуск

Пошаговая инструкция вынесена в файл **PRODUCTION_LAUNCH.md** в корне проекта. Краткий чек-лист:

1. Docker Desktop + PostgreSQL + Git установлены.
2. Проект на диске, ветка `production`.
3. `tileserver/data/map.mbtiles` на месте.
4. `backend/.env` из `.env.example`, `DEBUG=False`.
5. `docker load` образов → `import-and-start.ps1` или `compose up` с `server-override`.
6. `createsuperuser`, проверка URL `:5173` / `:8000` / `:8080`.

3. Повседневные операции

3.1. Старт / останов

```
cd D:\InfoLake\Lake_grok
docker compose -f docker-compose.yml -f docker-compose.server.yml up -d --no-build --pull
never
docker compose -f docker-compose.yml -f docker-compose.server.yml down
```

3.2. Статус и логи

```
docker compose -f docker-compose.yml -f docker-compose.server.yml ps
docker compose -f docker-compose.yml -f docker-compose.server.yml logs --tail 100
docker compose -f docker-compose.yml -f docker-compose.server.yml logs -f frontend
```

3.3. Проверка доступности

- `http://localhost:5173` — интерфейс
- `http://localhost:8000/admin/` — админка
- `http://localhost:8000/api/v1/` — API
- `http://localhost:8080/` — TileServer

На офлайн-сервере **запрещено**: `docker compose build`, `docker compose pull`, `docker pull`. Образы приходят только через `docker load` / пакет обновления.

4. Обновления

4.1. Для оператора

1. Получить от разработчика `infolake_update_vX.Y.Z.zip`.
2. Распаковать в любую папку (например `D:\InfoLake\updates\v1.0.1`).
3. Дважды щёлкнуть **apply-update.bat**.
4. Дождаться зелёного баннера «ОБНОВЛЕНИЕ ЗАВЕРШЕНО».

Скрипт автоматически:

- создаёт резервную копию PostgreSQL в `backups\`;
- применяет код из `update.bundle` (fast-forward ветки production);
- загружает Docker-образы из `images.tar`;
- перезапускает контейнеры, выполняет migrate и seed шаблонов отчётов;
- проверяет HTTP-доступность frontend / backend / tileserver.

4.2. При ошибке

Красный баннер «ОШИБКА — ВЫПОЛНЕН ОТКАТ» означает: скрипт вернул код и образы к состоянию до обновления. **Не пытайтесь чинить вручную.** Отправьте разработчику файл `logs\update-YYYYMMDD_HHMMSS.log`.

4.3. Для разработчика (сборка пакета)

```
git checkout production
# после merge из develop_report и bump VERSION / CHANGELOG
.\scripts\production\prepare-update-package.ps1
```

Результат: `dist\updates\infolake_update_vX.Y.Z.zip`.

5. Резервные копии и восстановление

5.1. Автобэкап перед обновлением

`apply-update.ps1` вызывает `scripts\offline\backup-postgres-before-migrate.ps1` → файл `backups\infolake_pre_migrate_*.dump` (формат custom, `pg_dump -Fc`).

5.2. Ручной бэкап

```
.\scripts\offline\backup-postgres-before-migrate.ps1
```

5.3. Восстановление (только с участием разработчика / опытного админа)

```
docker compose -f docker-compose.yml -f docker-compose.server.yml down
pg_restore -h localhost -U infolake -d infolake_db --clean --if-exists
backups\infolake_pre_migrate_XXXX.dump
docker compose -f docker-compose.yml -f docker-compose.server.yml up -d --no-build --pull
never
```

Автоматический откат БД скриптом обновления не выполняется. Откатываются код и Docker-образы; БД откатывают вручную из dump при необходимости.

Также сохраняйте копии `backend\media` и `tileserver\data\map.mbtiles`.

6. Пользователи и права

- Суперпользователь: `createsuperuser` (см. `PRODUCTION_LAUNCH.md`).
- Группы безопасности: Django Admin → группы; страны + уровни модулей (`none` / `read` / `write` / `write_delete`).
- Регистрация: статус `pending` → одобрение в «Управление пользователями».
- Сброс пароля: админ выдаёт временный пароль; пользователь обязан сменить его.
- Примеры групп: `seed_security_groups` .

Подробности API и JWT: `backend/docs/OFFLINE_AUTH.md` .

7. Файл `backend/.env`

Параметр	Production
<code>DEBUG</code>	False (обязательно)
<code>SECRET_KEY</code>	Длинная случайная строка, не из примера
<code>DB_*</code>	Учётные данные PostgreSQL на хосте
<code>DB_HOST</code>	<code>host.docker.internal</code> (Docker Desktop)
<code>ALLOWED_HOSTS</code>	<code>localhost, 127.0.0.1</code> или IP сервера

8. Страницы ошибок

- **Frontend 404** — неизвестный URL в браузере (не карта/логин).
- **Frontend ErrorBoundary** — сбой отрисовки React; кнопка «Обновить страницу».
- **Backend 404/500** — HTML-шаблоны при `DEBUG=False` (админка, редкие HTML-запросы к Django).

API по-прежнему отвечает JSON-ошибками (стандарт DRF).

9. Диагностика

Симптом	Действие
Карта пустая	Проверить наличие <code>tileserver/data/map.mbtiles</code> , логи <code>tileserver</code>
Backend не стартует / ошибка БД	<code>DB_HOST</code> , пароль, PostgreSQL запущен, firewall
Frontend «завис», <code>docker kill</code> не помогает	См. <code>OFFLINE_MIGRATION.md §8.1</code> : <code>wsl --shutdown</code> , лимиты памяти, <code>production compose</code>
<code>apply-update</code> : «не fast-forward»	Локальная история расходится — эскалация разработчику, не править <code>git</code> вручную
Медиа/фото 404 при <code>DEBUG=False</code>	Убедиться, что том <code>backend/media</code> на месте; код отдаёт <code>media</code> через Django

10. Эскалация разработчику

Перед обращением соберите:

1. Файл `logs\update-*.log` (если сбой обновления).
2. Вывод `docker compose ... ps` и `logs --tail 200` .
3. Содержимое `VERSION` и результат `git log -1 --oneline` .
4. Краткое описание: что делали, время, воспроизводится ли после `reboot`.

Связанные документы: `PRODUCTION_LAUNCH.md`, `OFFLINE_MIGRATION.md`, `OFFLINE_MIGRATION_MARKER_PALETTES.md`, `backend/docs/OFFLINE_AUTH.md`.